JavaScript Destructuring - Complete Notes

Table of Contents

Introduction to Destructuring

Destructuring is a JavaScript feature that allows you to extract values from arrays or properties from objects and assign them to variables in a single statement. It provides a clean and concise way to unpack values, making code more readable and efficient.

Why Use Destructuring?

- **Cleaner Code**: Reduces the need for repetitive property access
- **Improved Readability**: Makes variable assignments more explicit

- **Efficiency**: Allows multiple assignments in one line
- **Flexibility**: Works with nested structures and provides default values

---

Object Destructuring

Object destructuring allows you to extract properties from objects and assign them to variables with matching names.

Basic Syntax

```
const {property1, property2} = object;
```

Example from Class

```
const obj = {
    username: "Avinash",
    age: 24,
    city: "Noida"
}

const {username, age, city} = obj;

console.log(username); // "Avinash"
console.log(age);      // 24
console.log(city);     // "Noida"
```

Key Points:

- Variable names must match object property names
- Order doesn't matter in object destructuring
- Unmatched properties are ignored
- Missing properties result in `undefined`

Advanced Object Destructuring

Renaming Variables

```
const obj = {
    username: "Avinash",
    age: 24
}

const {username: name, age: userAge} = obj;
console.log(name);    // "Avinash"
console.log(userAge); // 24
```

Default Values

```
const obj = {
    username: "Avinash"
}

const {username, age = 25, city = "Delhi"} = obj;
console.log(username); // "Avinash"
console.log(age);      // 25 (default value)
console.log(city);     // "Delhi" (default value)
```

Array Destructuring.

Array destructuring assigns array elements to variables based on their position.

Basic Syntax

```
const [variable1, variable2] = array;
```

Example from Class

```
const movies = ["welcome", "housefull", "dhammal"];
const [m1, , m3] = movies; // Note: empty space skips second element

console.log(m1); // "welcome"
console.log(m3); // "dhammal"
```

Key Points:

- Position matters in array destructuring
- Use empty spaces to skip elements
- Excess variables become `undefined`
- Excess array elements are ignored

Advanced Array Destructuring

Rest Operator

```
const numbers = [1, 2, 3, 4, 5];
const [first, second, ...rest] = numbers;

console.log(first);  // 1
console.log(second); // 2
console.log(rest);   // [3, 4, 5]
```

Swapping Variables

```
let a = 1, b = 2;
[a, b] = [b, a]; // Swap values
console.log(a); // 2
console.log(b); // 1
```

Nested Object Destructuring

Destructuring works with nested objects, allowing you to extract deeply nested properties.

Example from Class

```
const obj = {
    username: "Avinash",
    address: {
        state: "UP",
        pin: 201301
    }
}

// Method 1: Rename nested object
const {username, address: add} = obj;
console.log(username); // "Avinash"
console.log(add);      // {state: "UP", pin: 201301}

// Method 2: Extract nested properties directly
const {username, address: {state, pin}} = obj;
console.log(username); // "Avinash"
console.log(state);    // "UP"
console.log(pin);      // 201301
```

Complex Nested Example

```
const user = {
    name: "Avinash",
    contact: {
        email: "avinashranjan918@gmail.com",
        phone: {
            mobile: "+91 6204732828",
            landline: "011-12345678"
        }
    }
}

const {
    name,
    contact: {
        email,
        phone: {mobile, landline}
    }
} = user;

console.log(name);     // "Avinash"
console.log(email);    // "avinashranjan918@gmail.com"
console.log(mobile);   // "+91 6204732828"
console.log(landline); // "011-12345678"
```

Nested Array Destructuring

Arrays can contain other arrays, and destructuring can extract elements from nested arrays.

Example from Class

```
const arr = [
    ["Html", "css"],
    ["js", "TS"],
    ["Node", "java"],
    ["Mongo", "sql"]
]

const [ui, logic, [b1, b2], db] = arr;

console.log(b1);    // "Node"
console.log(b2);    // "java"
console.log(ui);    // ["Html", "css"]
console.log(logic); // ["js", "TS"]
console.log(db);    // ["Mongo", "sql"]
```

Alternative Approach with Renaming

```
const arr = [
    ["Html", "css"],
    ["js", "TS"],
    ["Node", "java"],
    ["Mongo", "sql"]
]

const [ui, logic, backend = [b1, b2], db] = arr;
```

Mixed Array Destructuring

Arrays can contain objects, and destructuring can handle mixed data structures effectively.

Example from Class - Basic Approach

```
const users = [
    {
        fname: "avinash",
        lname: "ranjan   "
    },
    {
        fname: " Tinku  ",
        lname: "      sharma"
    },
    {
        fname: "Golu     ",
        lname: "verma  "
    }
];

// Traditional approach with separate destructuring
const x = users.map((element, index, array) => {
    const {fname, lname} = element;
    return {
        fname: fname.trim(), // removes extra spaces
        lname: lname.trim()
    }
})

console.log(users); // Original array (unchanged)
console.log(x);     // New array with trimmed names
```

Advanced Approach - Direct Destructuring in Parameters

```
const users = [
    {
        fname: "avinash",
        lname: "ranjan   "
    },
    {
        fname: " Tinku  ",
        lname: "      sharma"
    },
    {
        fname: "Golu     ",
        lname: "verma  "
    }
];

// Advanced: Destructuring directly in function parameters
const x = users.map(({fname, lname}, index, array) => {
    // Modify original array elements
    array[index] = {
        fname: fname.trim(),
        lname: lname.trim()
    }

    console.log("After trim:", fname.trim());
    console.log("After trim:", lname.trim());
})
```

Understanding the Difference

- **First approach**: Creates a new array, original remains unchanged
- **Second approach**: Modifies the original array elements
- **Key insight**: Destructuring in function parameters is cleaner and more functional

---

Advanced Destructuring Techniques

Function Parameters Destructuring

```
// Object parameters
function greetUser({name, age, city = "Unknown"}) {
    console.log(`Hello ${name}, you are ${age} years old from ${city}`);
}

greetUser({name: "Avinash", age: 24, city: "Noida"});

// Array parameters
function processCoordinates([x, y, z = 0]) {
    console.log(`X: ${x}, Y: ${y}, Z: ${z}`);
}

processCoordinates([10, 20]); // Z defaults to 0
```

Computed Property Names

```
const key = "username";
const obj = {
    username: "Avinash",
    age: 24
}

const {[key]: name, age} = obj;
console.log(name); // "Avinash"
```

Dynamic Destructuring with Loops

```
const users = [
    {id: 1, name: "Avinash", role: "developer"},
    {id: 2, name: "Tinku", role: "designer"},
    {id: 3, name: "Golu", role: "tester"}
];

for (const {id, name, role} of users) {
    console.log(`User ${id}: ${name} is a ${role}`);
}
```

Module Import/Export with Destructuring

Based on your `logic.js` file, here's how destructuring works with ES6 modules:

Export File (logic.js)

```
const add = (a, b) => a + b;
const sub = (a, b) => a - b;
const greet = (user) => `Good Morning ${user}`;
const user = "Avinash";

export {add, sub, greet, user};

// This creates an object-like structure:
// {
//     add: (a,b) => a + b,
//     sub: (a,b) => a - b,
//     greet: (user) => `Good Morning ${user}`,
//     user: "Avinash"
// }
```

Import with Destructuring

```
import {add, greet, user} from "./logic.js";

console.log(user);           // "Avinash"
console.log(add(5, 3));      // 8
console.log(greet("Tinku")); // "Good Morning Tinku"
```

Alternative Import Methods

```
// Import all as object
import * as utils from "./logic.js";
console.log(utils.add(5, 3));

// Import with renaming
import {add as addition, user as username} from "./logic.js";

// Import default and named exports
import defaultExport, {add, sub} from "./logic.js";
```

Best Practices

1. Use Meaningful Variable Names

```
// Good
const {firstName, lastName, email} = user;

// Avoid
const {a, b, c} = user;
```

2. Provide Default Values

```
const {name = "Anonymous", age = 0} = user;
```

3. Don't Over-Destructure

```
// Good - reasonable depth
const {user: {profile: {name}}} = data;

// Avoid - too deep, hard to read
const {user: {profile: {settings: {theme: {color}}}}} = data;
```

4. Use Rest Operator Wisely

```
// Good for separating first few items from the rest
const [first, second, ...others] = items;

// Good for extracting specific properties
const {id, name, ...otherProps} = user;
```

5. Handle Undefined Safely

```
// Safe destructuring with default empty object
const {name, age} = user || {};

// Or use optional chaining (ES2020)
const {name, age} = user?.profile || {};
```

6. Combine with Modern JavaScript Features

```
// With template literals
const {name, city} = user;
console.log(`${name} lives in ${city}`);

// With arrow functions
const getFullName = ({firstName, lastName}) => `${firstName} ${lastName}`;

// With async/await
const {data, error} = await fetchUser(id);
```

Summary

Destructuring is a powerful JavaScript feature that:

- **Simplifies variable assignment** from objects and arrays
- **Improves code readability** and reduces repetition
- **Works with nested structures** for complex data extraction
- **Supports default values** and variable renaming
- **Integrates seamlessly** with modern JavaScript features
- **Essential for modern JavaScript development**, especially with frameworks like React

Master destructuring to write cleaner, more maintainable JavaScript code!